

DBMS Technical Task

HEMAVATHI.B

E25AI114

B.Sc. Computer Science with Artificial Intelligence

Week 4: Customer Order Management and Order History System

Aim of the Project

After a customer places an order, the system stores the overall order information and the details of the individual products included in that order.

The **Orders** table stores the main information about each order, including the customer who placed the order, order date, total amount, order status, and shipping information.

The **Order_Details** table stores the products included in each order along with their quantity and price. Since an order can contain multiple products and a product can be included in multiple orders, the Order_Details table is used to establish the relationship between Orders and Products.

Create Orders and Order_Details Tables

Orders Table

The Orders table stores the main information about orders placed by customers on the e-commerce platform.

```
SQL> CREATE TABLE Orders(Order_ID VARCHAR(7),Customer_ID VARCHAR(7),Order_Date DATE,Total_Amount NUMBER(18,2),Order_Status VARCHAR(18));  
Table created.
```

Order_Details Table

The Order_Details table stores information about individual products included in an order.

```
SQL> CREATE TABLE ORDER_DETAILS(Order_Detail_ID VARCHAR2(7),Order_ID VARCHAR2(7),Product_ID VARCHAR2(7),Quantity NUMBER);  
Table created.
```

Establishing Relationships

After creating the Orders and Order Details tables, relationships were established between the tables using foreign keys. The Customer_ID in the Orders table refers to the corresponding Customer in the Customer table. Similarly, In Order Details Table Order_id connects the Orders table and Product_id connects the Product table Using foreign key .

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_ORDER FOREIGN KEY(Order_ID) REFERENCES ORDERS(Order_ID);  
Table altered.
```

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_ORDERS PRIMARY KEY(Order_ID);  
Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_ORDER_DETAILS PRIMARY KEY(Order_Detail_ID);  
Table altered.
```

Applying Constraints for Data Integrity

Applied to make sure that the database accepts only valid and meaningful data.

```
SQL> ALTER TABLE ORDER_DETAILS MODIFY(Order_ID NOT NULL,Product_ID NOT NULL);  
Table altered.
```

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_STATUS CHECK(Order_Status IN('Pending','Shipped','Delivered'));  
Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_QUANTITY CHECK(Quantity>0);  
Table altered.
```

Inserting Values

```
SQL> INSERT INTO Orders VALUES ('ORD001','CUS001',TO_DATE('25-09-2026','DD-MM-YYYY'),2500.00,'Delivered');
1 row created.

SQL> INSERT INTO Orders VALUES ('ORD002','CUS002',TO_DATE('25-09-2026','DD-MM-YYYY'),1800.00,'Pending');
1 row created.

SQL> INSERT INTO Orders VALUES ('ORD003','CUS003',TO_DATE('24-09-2026','DD-MM-YYYY'),3500.00,'Shipped');
1 row created.

SQL> INSERT INTO Orders VALUES ('ORD004','CUS004',TO_DATE('23-09-2026','DD-MM-YYYY'),1200.00,'Delivered');
1 row created.

SQL> INSERT INTO Orders VALUES ('ORD005','CUS005',TO_DATE('22-09-2026','DD-MM-YYYY'),4500.00,'Pending');
1 row created.
```

```
SQL> INSERT INTO Order_details VALUES ('OD001','ORD001','PROD001',2);
1 row created.

SQL> INSERT INTO Order_details VALUES ('OD002','ORD002','PROD002',1);
1 row created.

SQL> INSERT INTO Order_details VALUES ('OD003','ORD003','PROD003',3);
1 row created.

SQL> INSERT INTO Order_details VALUES ('OD004','ORD004','PROD004',1);
1 row created.

SQL> INSERT INTO Order_details VALUES ('OD005','ORD005','PROD005',2);
1 row created.
```

Report 1: Customer Order History

Displays:

- Customer Name
- Order ID
- Order Date

- Total Amount
- Order Status

```
SQL> SELECT c.CUSTOMER_ID,c.FIRST_NAME,c.LAST_NAME,o.ORDER_ID,o.ORDER_DATE,o.TOTAL_AMOUNT,o.ORDER_STATUS,od.PRODUCT_ID,od.QUANTITY FROM Customer c JOIN Orders o ON c.CUSTOMER_ID=o.CUSTOMER_ID JOIN Order_details od ON o.ORDER_ID=od.ORDER_ID;
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	ORDER_STATUS	PRODUCT_ID	QUANTITY
C-1801	Chiru	Raju	ORD001	25-SEP-26	2500	Delivered	BR-01	1
C-1802	Hema	Bhaskar	ORD002	25-SEP-26	1800	Pending	BR-02	1
C-1803	Seetha	Mani	ORD003	24-SEP-26	3500	Shipped	BR-03	3
C-1804	Abhinaya	Ramesh	ORD004	23-SEP-26	1200	Delivered	BR-04	1
C-1805	Deepikaa	Selvam	ORD005	22-SEP-26	4500	Pending	BR-05	2

Report 2: Product-wise Order Report

- Number of times each product was ordered.
- Total quantity of each product sold.

```
SQL> SELECT PRODUCT_ID,COUNT(*) AS TIMES_ORDERED,SUM(QUANTITY) AS TOTAL_QUANTITY_SOLD FROM Order_details GROUP BY PRODUCT_ID ORDER BY PRODUCT_ID;
```

PRODUCT_ID	TIMES_ORDERED	TOTAL_QUANTITY_SOLD
BR-01	1	2
BR-02	1	1
BR-03	1	3
BR-04	1	1
BR-05	1	2

Report 3: Customer Purchase Analysis

```
SQL> SELECT c.CUSTOMER_ID,c.FIRST_NAME||' '||c.LAST_NAME AS CUSTOMER_NAME,o.ORDER_ID,o.ORDER_DATE,od.PRODUCT_ID,od.QUANTITY,b.PRICE AS UNIT_PRICE,od.QUANTITY*b.PRICE AS SUBTOTAL FROM Customer c JOIN Orders o ON c.CUSTOMER_ID=o.CUSTOMER_ID JOIN Order_details od ON o.ORDER_ID=od.ORDER_ID JOIN Brand b ON od.PRODUCT_ID=b.PRODUCT_ID ORDER BY c.CUSTOMER_ID,o.ORDER_DATE;
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DATE	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL
C-1801	Chiru Raju	ORD001	25-SEP-26	BR-01	1	1500	3000
C-1802	Hema Bhaskar	ORD002	25-SEP-26	BR-02	1	1800	1800
C-1803	Seetha Mani	ORD003	24-SEP-26	BR-03	3	1200	3600
C-1804	Abhinaya Ramesh	ORD004	23-SEP-26	BR-04	1	2200	2200
C-1805	Deepikaa Selvam	ORD005	22-SEP-26	BR-05	2	2800	5600

Customers with Maximum Orders:

```
SQL> SELECT c.CUSTOMER_ID,c.FIRST_NAME||' '||c.LAST_NAME AS CUSTOMER_NAME,COUNT(o.ORDER_ID) AS ORDER_COUNT FROM Customer c JOIN Orders o ON c.CUSTOMER_ID=o.CUSTOMER_ID GROUP BY c.CUSTOMER_ID,c.FIRST_NAME,c.LAST_NAME HAVING COUNT(o.ORDER_ID)=(SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(*) AS ORDER_COUNT FROM Orders GROUP BY CUSTOMER_ID));
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_COUNT
C-1001	Chinu Raju	1
C-1002	Hema Bhaskar	1
C-1005	Deepika Selvam	1
C-1003	Seetha Mani	1
C-1004	Abinaya Ramesh	1

Total Spending by Each Customer:

```
SQL> SELECT c.CUSTOMER_ID,c.FIRST_NAME||' '||c.LAST_NAME AS CUSTOMER_NAME,SUM(o.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Customer c JOIN Orders o ON c.CUSTOMER_ID=o.CUSTOMER_ID GROUP BY c.CUSTOMER_ID,c.FIRST_NAME,c.LAST_NAME ORDER BY c.CUSTOMER_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_SPENDING
C-1001	Chinu Raju	2500
C-1002	Hema Bhaskar	1800
C-1003	Seetha Mani	3500
C-1004	Abinaya Ramesh	1200
C-1005	Deepika Selvam	4500

Average Order Value:

```
SQL> SELECT c.CUSTOMER_ID,c.FIRST_NAME||' '||c.LAST_NAME AS CUSTOMER_NAME,AVG(o.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM Customer c JOIN Orders o ON c.CUSTOMER_ID=o.CUSTOMER_ID GROUP BY c.CUSTOMER_ID,c.FIRST_NAME,c.LAST_NAME ORDER BY c.CUSTOMER_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	AVERAGE_ORDER_VALUE
C-1001	Chinu Raju	2500
C-1002	Hema Bhaskar	1800
C-1003	Seetha Mani	3500
C-1004	Abinaya Ramesh	1200
C-1005	Deepika Selvam	4500